DEPARTMENT OF SOFTWARE ENGINEERING

Quaid-e-Awam University of Engineering, Science & Technology Nawabshah

PRACTICAL WORKBOOK

for the subject

CLOUD COMPUTING

NAME:_____

ROLL: _____

DEPARTMENT OF SOFTWARE ENGINEERING
Quaid-e-Awam University of Engineering, Science & Technology Nawabshah

Practical Work

Course:	Cloud Computing	**Course Code:**	SW406
Teacher Name:	Prof. Dr. Pardeep Kumar	**Department:**	Software Engineering
Semester, Year:	1st Semester, 4th Year	**Batch:**	20SW

Class Learning Outcomes (CLOs)

SN	Description	PLO	Level	Assessment (50)		
				Assignment (10)	Objective (20)	Viva (20)
1	**Practice** working with key cloud platforms, devices, and technologies	3	A (3)	10	05	10
2	**Build** cloud networks using various configurations, platforms, and tools	4	P (3)	10	05	10

Practical Objectives

		Associated CLO	Hours	Date
1	To get understanding about cloud computing technologies, tools and different platforms	**1**	3	
2	To work with Remote Method Invocation (RMI) API and Object Serialization	**2**	3	
3	To work with Remote Method Invocation (RMI) Callbacks	**2**	3	
4	To work with RMI-IIOP	**2**	3	
5	To implement multithreading	**2**	3	
6	To work with MVC architecture for Enterprise level applications	**2**	3	
7	To implement preserving user state in distributed applications	**2**	3	
8	To work with Spring framework	**2**	3	
9	To work with JDBC template of Spring framework	**2**	3	
10	To work with cloud computing using Microsoft Azure	**2**	3	
11	To work with Amazon web services (AWS)	**2**	3	
12	Develop distributed applications using Amazon web services	**2**	3	
13	To become familiar with Amazon's AWS Identity Management and Security in the Cloud	**2**	3	
14	To make Amazon's RESTFul WebServices	**2**	3	
15	Case study /Open-ended lab	**2**	3	

Name & Signature of Teacher: _____ Date: _____

Practical 01

Objective: Working with Remote Method Invocation (RMI) API

Description: In this lab, students will delve into the world of distributed computing by mastering the Remote Method Invocation (RMI) API in Java. They will learn to create distributed applications where objects can invoke methods on remote systems seamlessly. Through hands-on exercises, students will understand the intricacies of stubs, skeletons, and remote object registration, enabling them to build robust distributed systems.

Introduction to RMI and its Significance:

Remote Method Invocation (RMI) is a Java API that allows objects in one Java Virtual Machine (JVM) to invoke methods on objects in another JVM, enabling distributed computing. It provides a mechanism for communication between Java applications running on different machines over a network.

Role in Distributed Computing:

Distributed computing involves breaking down tasks into smaller parts and distributing them across multiple computers connected via a network. RMI plays a crucial role in distributed computing by facilitating communication between distributed components or objects in a Java environment.

Traditionally, distributed systems relied on technologies like sockets or web services for communication between components. While these approaches are effective, they often require developers to handle low-level networking details, serialization, and marshalling of data. RMI abstracts away these complexities, providing a high-level interface for remote communication.

Importance for Developing Distributed Java Applications:

RMI is essential for developing distributed Java applications due to several reasons:

1. **Simplicity**: RMI simplifies the development of distributed Java applications by providing a high-level abstraction for remote communication. Developers can focus on defining interfaces and implementing business logic, while RMI handles the underlying communication details.
2. **Java-centric Approach**: RMI is native to the Java programming language, making it well-integrated with other Java technologies and frameworks. This seamless integration allows developers to leverage existing Java libraries and tools, streamlining the development process.
3. **Object-oriented Paradigm**: RMI follows the object-oriented paradigm, allowing remote objects to be treated like local objects. This abstraction enables developers to write distributed applications in a natural and intuitive manner, without the need for extensive networking code.
4. **Efficiency**: RMI optimizes communication between distributed Java applications by using efficient protocols and serialization mechanisms. This efficiency ensures that remote method invocations are performed with minimal overhead, leading to better performance and scalability.
5. **Security**: RMI provides built-in support for security features such as codebase and permission management, allowing developers to secure their distributed applications against unauthorized access and malicious attacks.

Explanation of Key Concepts:

1. **Stubs:**
   - **Definition**: A stub is a client-side proxy for a remote object. It resides in the client's JVM and acts as a local representative of the remote object.
   - **Role**: When a client invokes a method on the stub, the stub marshals the method parameters, sends them to the remote object through the network, and receives the result back from the remote object.
   - **Example**: Suppose we have a remote interface RemoteInterface with a method remoteMethod(). The stub for this remote interface would have a method remoteMethod() that internally handles the network communication and delegates the method invocation to the remote object.
2. **Skeletons:**
   - **Definition**: A skeleton is a server-side object that dispatches incoming remote method calls to the appropriate remote object implementation. It resides in the server's JVM.
   - **Role**: The skeleton receives remote method calls from clients, unmarshals the method parameters, and invokes the corresponding methods on the actual remote object.
   - **Example**: Continuing from the previous example, the skeleton for the RemoteInterface would have a method remoteMethod() that receives the method call from the stub, unmarshals the parameters, and invokes the actual implementation of remoteMethod() on the server.
3. **Remote Object Registration:**
   - **Definition**: Remote object registration involves binding a remote object to a naming registry, such as the Java RMI Registry. This registration allows clients to look up the remote object by its name and obtain a reference to invoke its methods.
   - **Role**: Remote object registration enables clients to locate and communicate with remote objects across the network by providing a centralized naming service.
   - **Example**: In RMI, the java.rmi.registry.Registry interface provides methods like bind() and lookup() to register and lookup remote objects, respectively. For example, registry.bind("RemoteObject", remoteObject) registers a remote object named "RemoteObject" with the registry.

Overview of RMI Architecture:

The RMI architecture consists of the following components:

1. **Client**:
   - The client is the Java application that initiates remote method invocations on remote objects.
   - It interacts with stubs to communicate with remote objects running on the server.
2. **Server**:
   - The server is the Java application that hosts remote objects and provides services to clients.
   - It interacts with skeletons to handle incoming remote method invocations from clients and delegate them to the appropriate remote object implementations.
3. **Stubs and Skeletons**:
   - Stubs and skeletons act as intermediaries between clients and servers, facilitating communication by marshalling and unmarshalling method parameters and managing the flow of remote method invocations.

4. **Java RMI Registry**:
   - The Java RMI Registry is a simple server-side bootstrap naming facility that enables clients to obtain references to remote objects.
   - It acts as a lookup service where remote objects can be registered and looked up by their names, enabling clients to locate and communicate with remote objects.

In RMI architecture, clients interact with stubs to invoke methods on remote objects, while servers use skeletons to handle incoming method invocations and interact with the actual remote object implementations. The Java RMI Registry provides a centralized naming service for registering and looking up remote objects, facilitating communication between distributed Java applications.

Practical Implementation:

1. Setting up the Development Environment

Installing JDK:

- Download the latest version of JDK from the official Oracle website or another trusted source.
- Follow the installation instructions for your operating system (Windows, macOS, or Linux).
- Set the JAVA_HOME environment variable to point to the JDK installation directory.

Configuring the IDE:

- Install an Integrated Development Environment (IDE) such as IntelliJ IDEA, Eclipse, or NetBeans.
- Open the IDE and configure it to use the installed JDK for Java development.
- Install any necessary plugins or extensions for RMI development (if required).

Creating a Simple RMI Application

Step 1: Define a Remote Interface

- Create a new Java interface that extends java.rmi.Remote.
- Declare the remote methods that will be invoked by clients.

Example:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface RemoteInterface extends Remote {
   String sayHello() throws RemoteException;
}
```

Step 2: Implement the Remote Interface

- Create a Java class that implements the remote interface.
- Implement the methods declared in the remote interface.

Example:

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class RemoteObject extends UnicastRemoteObject implements RemoteInterface {
   public RemoteObject() throws RemoteException {
super();
   }
   @Override
   public String sayHello() throws RemoteException {
```

```
        return "Hello, World!";
    }
}
```

Step 3: Write a Server Program

- Create a Java class that contains the main method to start the RMI server.
- Create an instance of the remote object and export it using java.rmi.registry.LocateRegistry.
- Bind the remote object to the RMI Registry using registry.bind().

Example:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Server {
    public static void main(String[] args) throws Exception {
        Registry registry = LocateRegistry.createRegistry(1099);
registry.bind("RemoteObject", new RemoteObject());
System.out.println("Server started!");
    }
}
```

Step 4: Write a Client Program

- Create a Java class that contains the main method to invoke remote methods.
- Obtain a reference to the remote object from the RMI Registry using registry.lookup().
- Invoke remote methods on the remote object.

Example:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {
    public static void main(String[] args) throws Exception {
        Registry registry = LocateRegistry.getRegistry("localhost", 1099);
RemoteInterfaceremoteObject = (RemoteInterface) registry.lookup("RemoteObject");
System.out.println(remoteObject.sayHello());
    }
}
```

Note: Ensure that both the server and client programs are running simultaneously for successful communication between them. Additionally, handle exceptions appropriately in your code to ensure robustness and reliability.

Conclusion:

In this lab, we delved into the world of Remote Method Invocation (RMI) in Java, exploring its significance in distributed computing and understanding key concepts such as stubs, skeletons, and remote object registration. We also gained insights into the RMI architecture and how its components work together to enable communication between distributed Java applications.

Throughout the lab, we learned that RMI provides a simplified approach to developing distributed Java applications by abstracting away low-level networking details and providing a high-level interface for remote communication. Its Java-centric approach, object-oriented paradigm, efficiency, and built-in security features make it essential for developing robust and scalable distributed systems.

In the practical implementation section, we set up the development environment by installing JDK and configuring the IDE for Java development. We then created a simple RMI application by defining a remote interface, implementing it to create a remote object, writing a server program to export the remote object and bind it to the RMI Registry, and developing a client program to interact with the remote object.

LAB TASKS:

1. Now that you have gained a foundational understanding of RMI and implemented a simple RMI application, your task is to enhance the application by adding additional remote methods to the remote interface and implementing them in the remote object. These methods could perform various tasks or calculations, such as arithmetic operations, string manipulation, or data processing.
2. Additionally, explore how you can handle exceptions gracefully in your RMI application to ensure robustness and reliability. Modify your code to handle exceptions such as RemoteException and other specific exceptions that may arise during remote method invocations or network communication.
3. Finally, experiment with deploying your RMI application on multiple machines or virtual environments to simulate a real-world distributed environment. Test the communication between the server and clients across different machines and observe how RMI facilitates seamless interaction between distributed Java applications.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

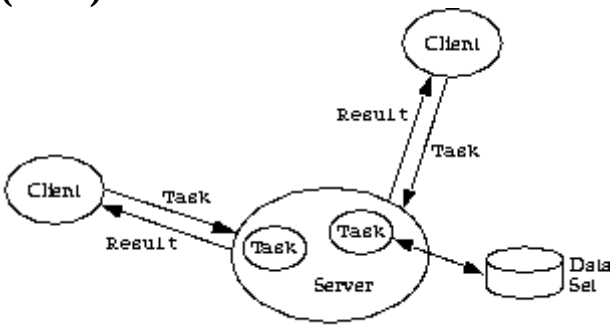
Practical 02

Objective: Working with Remote Method Invocation (RMI) Object Serialization

Description: This lab dives into the core of distributed computing with Java's Remote Method Invocation (RMI) by exploring object serialization. Students will learn how to serialize Java objects to facilitate their transmission across the network, enabling seamless communication between client and server applications. Through practical exercises, students will gain a deep understanding of serialization techniques and their role in distributed systems.

Introduction to Remote Method Invocation (RMI):

Remote Method Invocation (RMI) is a Java API that enables distributed computing by allowing a Java program to invoke methods on remote objects running in different Java Virtual Machines (JVMs). RMI provides a mechanism for communication between client and server applications in a distributed environment.



Key points about RMI:

1. **Role in Distributed Computing**: RMI facilitates communication between distributed components of a Java application by allowing objects to invoke methods on objects residing in remote JVMs. It abstracts away the complexities of network communication and provides a seamless way for distributed components to interact.
2. **Remote Objects**: In RMI, remote objects are instances of Java classes that can be accessed remotely by clients. These objects are hosted in a server JVM and can receive method invocations from client JVMs over the network.
3. **Communication between Client and Server**: RMI uses Java's serialization mechanism to pass objects between client and server JVMs. Parameters and return values of remote method invocations are serialized into byte streams and transmitted over the network.
4. **Interface Definition**: RMI requires remote objects to implement a remote interface that extends the java.rmi.Remote interface. This interface declares the methods that can be invoked remotely.
5. **RMI Registry**: RMI uses a registry service to bind remote objects with names, allowing clients to locate and invoke methods on these objects.
6. **Security**: RMI provides support for security through Java's security model, including mechanisms such as code signing, authentication, and access control.

Object Serialization:

Object serialization is the process of converting a Java object into a byte stream that can be transmitted over a network or stored persistently. Serialization is important in distributed systems for transmitting objects between different JVMs.

Key points about object serialization:

1. **Conversion to Byte Streams**: Serialization allows Java objects to be converted into byte streams, which can then be transmitted over a network or saved to a file or database.
2. **Transmitting Java Objects**: Serialization enables the transmission of complex Java objects, including their state and structure, between different JVMs.

3. **Serialization Mechanism**: Java's serialization mechanism is built into the language and runtime. It automatically handles the serialization and deserialization of objects, including handling object graphs, references, and complex data structures.
4. **Serializable Interface**: The java.io.Serializable interface is a marker interface that indicates that a Java class can be serialized. Classes that implement this interface can have their instances serialized and deserialized.

Implementing Serializable:

To implement the Serializable interface in Java classes:
Ensure that the class implements the java.io.Serializable interface. This is a marker interface and does not contain any methods.

```
import java.io.Serializable;

public class MyClass implements Serializable {
    // class definition
}
```

Serialization and Deserialization:
Serialization: Use ObjectOutputStream to serialize objects into byte streams.

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;

// Serialization
MyClassobj = new MyClass();
FileOutputStreamfileOut = new FileOutputStream("object.ser");
ObjectOutputStream out = new ObjectOutputStream(fileOut);
out.writeObject(obj);
out.close();
fileOut.close();
```

Deserialization: Use `ObjectInputStream` to deserialize byte streams back into objects.

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;

// Deserialization
FileInputStreamfileIn = new FileInputStream("object.ser");
ObjectInputStream in = new ObjectInputStream(fileIn);
MyClassnewObj = (MyClass) in.readObject();
in.close();
fileIn.close();
```

Best Practices:

- Ensure that all fields of a serializable class are themselves serializable.
- Avoid serializing sensitive or transient data.
- Implement the serialVersionUID field to provide versioning control.
- Consider customizing serialization using writeObject() and readObject() methods for complex classes.

In summary, object serialization is crucial in distributed systems for transmitting Java objects between different JVMs. The Serializable interface enables serialization for Java classes, and serialization and

deserialization are performed using ObjectOutputStream and ObjectInputStream classes. Best practices include ensuring field serializability and version control.

RMI and Serialization:

Integrating object serialization with RMI allows for the transmission of Java objects between client and server applications in a distributed environment. Here's an overview of the steps involved in RMI communication with serialized objects:

1. **Define Remote Interface**: Create a remote interface that extends java.rmi.Remote and declares the methods that can be invoked remotely.
2. **Implement Remote Object**: Implement the remote interface in a class that extends java.rmi.server.UnicastRemoteObject and implements the methods declared in the remote interface.
3. **Serialize Objects**: Ensure that any custom Java objects used as parameters or return values in remote method invocations are serializable by implementing the java.io.Serializable interface.
4. **Server Side Setup**: Start an RMI registry and bind the remote object implementation to a name in the registry using java.rmi.registry.LocateRegistry and java.rmi.registry.Registry classes.
5. **Client Side Setup**: Obtain a reference to the remote object from the RMI registry using java.rmi.registry.LocateRegistry and java.rmi.registry.Registry classes.
6. **Invoke Remote Methods**: Use the obtained reference to invoke remote methods on the server object, passing serialized Java objects as parameters and handling serialized return values.

Hands-on Exercises:

Practical exercises for implementing RMI communication with serialized objects could include:

1. Creating a simple client-server application using RMI.
2. Defining a remote interface with remote methods.
3. Implementing a server class that implements the remote interface and extends java.rmi.server.UnicastRemoteObject.
4. Implementing a client class that looks up the remote object in the RMI registry and invokes remote methods.
5. Serializing custom Java objects as parameters and return values in remote method invocations.

Error Handling and Troubleshooting:

Common errors and exceptions related to object serialization and RMI communication include:

1. java.io.NotSerializableException: This exception occurs when attempting to serialize an object that does not implement the java.io.Serializable interface.
2. java.rmi.UnmarshalException: This exception occurs when there is a problem deserializing an object received from the network during RMI communication.
3. java.rmi.RemoteException: This exception indicates a failure in the RMI communication between client and server.

Strategies for troubleshooting serialization issues in distributed systems include:

1. Ensuring that all custom Java objects used in RMI communication are serializable by implementing the java.io.Serializable interface.
2. Checking for proper handling of serialization and deserialization in the client and server code.
3. Verifying that the RMI registry is running and accessible to both client and server applications.
4. Debugging the code to identify any specific serialization or RMI communication issues.

Conclusion and Recap:

Throughout this lab, we have covered several key concepts related to integrating object serialization with Remote Method Invocation (RMI) for distributed computing in Java. Let's summarize the key points:

1. **Remote Method Invocation (RMI):** RMI is a Java API that allows Java objects to invoke methods on remote objects running in different Java Virtual Machines (JVMs). It facilitates communication between client and server applications in a distributed environment.
2. **Object Serialization**: Object serialization is the process of converting a Java object into a byte stream that can be transmitted over a network or stored persistently. In the context of RMI, serialization is crucial for passing Java objects between client and server JVMs during method invocations.
3. **Integration of Serialization with RMI**: When using RMI, Java objects passed between client and server JVMs during method invocations must be serializable. This is achieved by implementing the java.io.Serializable interface in the classes of these objects. Serialization allows for the seamless transmission of complex Java objects between distributed components.
4. **Importance of Object Serialization in RMI**: Object serialization plays a critical role in RMI-based distributed computing by enabling the transmission of Java objects between client and server applications. It ensures that the state of objects, including their data and structure, can be accurately replicated across different JVMs, thus maintaining the integrity of Distributed systems.

LAB TASK:

1. Implement Remote Method Invocation (RMI) communication with object serialization in Java.

   Description: In this task, you will create a simple client-server application using RMI, where Java objects will be serialized and transmitted between the client and server applications.

Requirements:

- JDK (Java Development Kit) installed on your system.
- Basic understanding of Java programming language and object-oriented concepts.
- Text editor or IDE (Integrated Development Environment) such as IntelliJ IDEA or Eclipse.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 03

Objective: To work with Remote Method Invocation (RMI) Callbacks

Description: Remote Method Invocation (RMI) is a Java API that performs the object-oriented equivalent of remote procedure calls (RPCs), allowing a Java object to invoke methods on an object running in another Java virtual machine (JVM). RMI Callbacks enable asynchronous communication between distributed Java applications, allowing a client to register callback methods on a server and receive notifications or responses from the server asynchronously.

How does Remote Method Invocation (RMI) Callbacks work?

RMI Callbacks facilitate asynchronous communication between distributed Java applications. Here's an overview of how RMI Callbacks work:

1. **Server Registration:** The server exposes remote methods that clients can invoke. Additionally, the server provides callback interfaces that clients can implement to receive asynchronous notifications.
2. **Client Callback Registration:** Clients register callback implementations with the server by passing instances of callback interfaces to remote methods during invocation.
3. **Asynchronous Invocation:** When certain events occur on the server or as a response to specific client requests, the server invokes callback methods on the registered client objects asynchronously.
4. **Callback Execution:** Client-side callback methods are executed upon receiving notifications from the server, allowing clients to handle events or process server responses asynchronously.

Step by Step Guide to Getting Started with RMI Callbacks

Getting started with RMI Callbacks involves understanding the principles of remote method invocation and asynchronous communication in Java. Here is a step-by-step guide to assist you with starting your journey with RMI Callbacks:

1: Understanding Remote Method Invocation (RMI)

- Familiarize yourself with the basic concepts of RMI, including remote interfaces, stubs, and skeletons.
- Learn how to define remote interfaces and implement remote objects that can be accessed by clients.

2: Implementing Callback Interfaces

- Define callback interfaces that extend the java.rmi.Remote interface to indicate that they are intended for remote invocation.
- Implement callback methods within these interfaces to specify the behavior that clients wish to execute asynchronously.

3: Registering Callbacks on the Server

- Expose remote methods on the server that allow clients to register callback implementations.
- Implement server-side logic to manage callback registrations and invoke callback methods as needed.

4: Invoking Remote Methods with Callbacks

- Clients invoke remote methods on the server and pass callback implementations as parameters to register for asynchronous notifications.
- Handle callback responses or notifications asynchronously within client applications.

Lab Task:

Task 1: Define a simple remote interface that includes methods for callback registration and invocation.
Task 2: Implement a server class that exposes remote methods for callback registration and invocation.
Task 3: Implement a client class that registers callback implementations with the server and invokes remote methods asynchronously.
Task 4: Test the RMI Callback functionality by triggering callback invocations on the server and observing the asynchronous execution of client-side callback methods.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 04

Objective: Working with Remote Method Invocation RMIIIOP:

Description: In this lab, students will dive into the realm of distributed computing by mastering the Remote Method Invocation (RMI) API integrated with Internet InterORB Protocol (IIOP) in Java. They will learn to create distributed applications that seamlessly invoke methods on remote systems across different ORBs using IIOP. Through practical exercises, students will grasp the nuances of RMIIIOP, including object marshalling, remote object registration, and interORB communication, empowering them to develop robust distributed systems.

Introduction to RMIIIOP and its Significance:

RMIIIOP (Remote Method Invocation over Internet InterORB Protocol) extends the capabilities of the RMI API by integrating it with the IIOP protocol, enabling interoperability between Java applications and other CORBAcompliant systems over the Internet. This integration facilitates seamless communication between distributed components or objects across different ORBs, enhancing the reach and flexibility of distributed computing solutions.

Role in Distributed Computing:

RMIIIOP plays a vital role in distributed computing by providing a standardized mechanism for communication between distributed Java applications and other CORBAcompliant systems. It abstracts away the complexities of interORB communication, allowing developers to focus on building distributed systems without worrying about underlying networking details.

Importance for Developing Distributed Java Applications:

1. **Interoperability:** RMIIIOP enables interoperability between Java applications and other CORBAcompliant systems, allowing seamless communication across heterogeneous environments.
2. **Scalability:** By leveraging the IIOP protocol, RMIIIOP enables scalable distributed solutions that can accommodate a large number of clients and handle increasing workloads efficiently.
3. **Reusability:** RMIIIOP promotes code reuse by allowing developers to leverage existing CORBA components and libraries in their Java applications, reducing development time and effort.
4. **Standardization:** RMIIIOP follows industrystandard protocols and specifications, ensuring compatibility and interoperability with other distributed computing technologies and platforms.
5. **Security:** RMIIIOP provides builtin support for security features such as authentication, authorization, and encryption, ensuring secure communication between distributed components.

Explanation of Key Concepts:

1. **Object Marshalling***:* RMIIIOP serializes Java objects into a format suitable for transmission over the network, allowing remote method invocations to be performed seamlessly across different ORBs.
2. **InterORB Communication***:* RMIIIOP facilitates communication between distributed Java applications and other CORBAcompliant systems by bridging the gap between disparate ORBs using the IIOP protocol.

3. **Portable Object Adapters (POAs):** POAs in RMIIIOP provide a flexible mechanism for managing distributed objects and their lifecycle, allowing developers to configure and control object behavior in distributed environments.
4. **Naming Services***:* RMIIIOP utilizes naming services such as the CosNaming service to register and locate remote objects across different ORBs, enabling clients to discover and interact with distributed components dynamically.

Overview of RMIIIOP Architecture:
The architecture of RMIIIOP consists of the following components:

1. **Java RMI:** The Java RMI API provides the foundation for building distributed Java applications and enables seamless integration with IIOP for interORB communication.

2. **IIOP:** The Internet InterORB Protocol (IIOP) serves as the standard protocol for communication between ORBs, enabling interoperability between Java applications and other CORBAcompliant systems.

3. **ORBs:** Object Request Brokers (ORBs) are middleware components responsible for managing distributed objects and facilitating communication between clients and servers in a distributed environment.

4. **Portable Object Adapters (POAs):** POAs provide a flexible and configurable framework for managing distributed objects in RMIIIOP, allowing developers to control object behavior and lifecycle in distributed systems.

Practical Implementation:
1. Setting up the Development Environment:
2. Install JDK and configure the IDE for Java development.
3. Ensure compatibility with CORBAcompliant systems by installing necessary libraries and extensions.
4. Creating a Simple RMIIIOP Application:
5. Define a remote interface extending java.rmi.Remote and org.omg.CORBA.Object.
6. Implement the remote interface to create a remote object with IIOP support.
7. Write a server program to export the remote object and bind it to the CORBA Naming Service.
8. Develop a client program to interact with the remote object using IIOP.

Conclusion:
In this lab, we explored the concepts of Remote Method Invocation (RMI) integrated with Internet InterORB Protocol (IIOP) in Java and its significance in distributed computing. We learned how RMIIIOP enables interoperability, scalability, and security in distributed Java applications, empowering developers to build robust and flexible distributed systems. Through practical exercises, we gained insights into the architecture and key concepts of RMIIIOP, paving the way for building advanced distributed computing solutions in Java.

LAB TASK:

1. Setup RMIIIOP Development Environment:

- Objective: Configure the development environment for RMIIIOP programming.
- Description: Install JDK, set up the IDE, and configure necessary libraries and extensions for developing RMIIIOP applications.

2. Implement Remote Interface with IIOP Support:

- Objective: Define a remote interface with IIOP support for RMIIIOP applications.
- Description: Create a Java interface extending java.rmi.Remote and org.omg.CORBA.Object to define remote methods for RMIIIOP communication.

3. Develop Server Program for RMIIIOP Application:

- Objective: Write a server program to export remote objects and bind them to the CORBA Naming Service.
- Description: Implement a Java class containing the main method to initialize the RMIIIOP server, export remote objects, and register them with the CORBA Naming Service.

4. Create Client Program for RMIIIOP Application:

- Objective: Develop a client program to interact with remote objects using IIOP.
- Description: Write a Java class with the main method to connect to the RMIIIOP server, look up remote objects from the CORBA Naming Service, and invoke remote methods.

5. Test and Debug RMIIIOP Application:

- Objective: Verify the functionality of the RMIIIOP application and debug any issues.
- Description: Execute the server and client programs, perform remote method invocations, and analyze the output. Debug any errors or exceptions encountered during execution.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 05

Objective: **Multithreading Practical Building Concurrent Java Applications**

Description: In this lab, Gain hands-on experience in implementing multithreading in Java, understand the basics of thread synchronization, and explore advanced features for concurrent programming.

1. Introduction to Multithreading:

Multithreading is the concurrent execution of two or more threads. In Java, threads are lightweight processes that share the same memory space, allowing developers to execute multiple tasks simultaneously.

2. Basic Multithreading Concepts:

Thread Creation - 'SimpleThreadExample':

```
class SimpleThread extends Thread {
   public void run() {
      for (int i = 0; i < 5; i++) {
         System.out.println(Thread.currentThread().getId() + " Value " + i);
      }
   }
}
public class SimpleThreadExample {
   public static void main(String args[]) {
      for (int i = 0; i < 3; i++) {
         new SimpleThread().start();
      }
   }
}
```

Runnable Interface - RunnableExample:

```
class SimpleRunnable implements Runnable {
   public void run() {
      for (int i = 0; i < 5; i++) {
         System.out.println(Thread.currentThread().getId() + " Value " + i);
      }
```

```
    }
}
public class RunnableExample {
    public static void main(String args[]) {
        Thread t1 = new Thread(new SimpleRunnable());
        Thread t2 = new Thread(new SimpleRunnable());
        t1.start();
        t2.start();
    }
}
```

3. Thread Synchronization:

Synchronized Method – 'SynchronizedMethodExample':

```
class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
    public synchronized int getCount() {
        return count;
    }
}
public class SynchronizedMethodExample {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
```

```
            counter.increment();
        }
    });
    Thread t2 = new Thread(() -> {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
    });
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println("Counter: " + counter.getCount());
  }
}
```

4. Advanced Multithreading Concepts:

Executor Framework - 'ExecutorFrameworkExample':

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
class SimpleTask implements Runnable {
  public void run() {
    System.out.println(Thread.currentThread().getId() + " Task is being
processed.");
  }
}
public class ExecutorFrameworkExample {
  public static void main(String[] args) {
```

```
        ExecutorService executor = Executors.newFixedThreadPool(5);
        for (int i = 0; i < 10; i++) {
            Runnable task = new SimpleTask();
            executor.execute(task);
        }
        executor.shutdown();
    }
}
```

Callable and Future - 'CallableFutureExample':

```
import java.util.concurrent.*;
class SimpleCallable implements Callable<String> {
    public String call() {
        return Thread.currentThread().getId() + " Task is being processed.";
    }
}
public class CallableFutureExample {
    public static void main(String[] args) throws ExecutionException,
InterruptedException {
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<String> future = executor.submit(new SimpleCallable());
        System.out.println(future.get());
        executor.shutdown();
    }
}
```

LAB TASKS:

Now that you have a basic understanding of multithreading in Java, enhance the application by:

1. **Producer-Consumer Problem:**

   Implement a simple producer-consumer problem using wait() and notify().

2. **Thread Pools:**

   Modify the Executor Framework example to use a thread pool with different types of Executors.

3. **Concurrency in Collections:**

   Explore and use concurrent collections like ConcurrentHashMap or CopyOnWriteArrayList.

4. **Advanced Synchronization:**

   Implement advanced synchronization techniques such as using ReentrantLock or Semaphore.

5. **Parallel Streams:**

   Explore Java 8 parallel streams for concurrent processing of collections.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 06

Objective: To Work with MVC Architecture for Enterprise Level Applications

Introduction to MVC

The ModelViewController (MVC) architecture is a design pattern that separates an application into three main components: Model, View, and Controller. This separation helps manage complex applications by dividing them into smaller, more manageable pieces.

Working with MVC Architecture for Enterprise Level Applications (Detailed)

This guide delves deeper into the details of utilizing the Model-View-Controller (MVC) architecture for building robust and scalable enterprise-level applications.

1. Deep Dive into Components:

1.1 Model:

- **Responsibilities:**
  - **Data Representation:** Defines classes encapsulating application data (e.g., Product, Order, User). These classes typically have attributes representing data fields and methods for accessing and manipulating data.
  - **Business Logic:** Implements core application logic and rules. This may involve data validation, calculations, and operations on the data objects (e.g., order discount calculation, user authentication).
  - **Data Access:** Interacts with persistent data storage mechanisms like databases or external APIs. Implement data access layers or repositories to manage connections, execute queries, and retrieve data.

1.2 View:

- **Responsibilities:**
  - **Presentation Layer:** Responsible for transforming and presenting data to the user in a human-readable format. This typically involves utilizing templating engines (e.g., Jinja2, Django templates, Razor) to dynamically generate HTML content.
  - **User Interaction:** Handles basic user interactions such as displaying forms, capturing user input, and triggering events. However, the view should **not** contain complex logic for processing user input or manipulating data.

1.3 Controller:

- **Responsibilities:**

- **Request Handling:** Acts as the central point for handling incoming user requests and directing the application flow. This involves routing requests based on URLs or other parameters.
- **Model Interaction:** Interacts with the model layer to retrieve or manipulate data as needed based on the request. This may involve invoking specific model methods for data access or business logic execution.
- **View Selection:** Selects the appropriate view to be displayed based on the request and data from the model.
- **Data Transfer:** Passes necessary data (typically extracted from the model) to the chosen view for presentation.

2. Benefits for Enterprise Applications (Detailed):

2.1 Maintainability:

- **Component Independence:** Each layer (Model, View, Controller) is independent, allowing developers to work on specific areas without impacting others. This facilitates easier code maintenance and reduces the risk of introducing side effects.
- **Code Reusability:** Components can be reused across different functionalities, reducing code duplication and improving development efficiency.
- **Testability:** Each layer can be tested independently, making testing more manageable and ensuring individual components function correctly before integration.

2.2 Scalability:

- **Horizontal Scaling:** By isolating layers, it becomes easier to scale specific components independently. For example, additional web servers can be added to handle increased user load on the controller layer.
- **Vertical Scaling:** Each layer can be optimized for its specific needs. For instance, the model layer might benefit from a more powerful database server for efficient data access.

2.3 Security:

- **Layered Security:** Business logic resides within the model, allowing for stricter control over data access and manipulation. This helps prevent unauthorized access to sensitive information.

- **Input Validation:** The model layer can implement robust validation rules for user input before processing it, mitigating potential vulnerabilities like SQL injection attacks.

3. Practical Implementation with Enterprise Considerations:

3.1 Detailed Implementation:

3.1.1 Model Layer:

- **Domain Objects:** Develop classes representing domain entities (e.g., `Product` with attributes like `id`, `name`, `price`).
- **Data Access Layer (DAL):** Implement a separate layer (e.g., repositories) responsible for interacting with the database or external APIs. This separates data access logic from the core business logic.
- **Business Logic Implementation:** Create methods within the model classes or separate service classes to encapsulate business logic (e.g., `calculateOrderTotal`, `validateUserCredentials`).

3.1.2 View Layer:

- **Templates:** Design HTML templates (e.g., using Jinja2, Django templates, Razor) to define the overall structure and layout of the user interface.
- **Data Display:** Use templating engine syntax to dynamically display data received from the controller (e.g., loop through a list of products to display them).
- **User Input Capture:** Use HTML form elements (e.g., text fields, dropdowns) to capture user input, but refrain from processing or validating it in the view.

3.1.3 Controller Layer:

- **Route Mapping:** Use framework-specific features (e.g., `@RequestMapping` in Spring MVC) to define mappings between URLs and controller methods.
- **Request Handling:** Implement methods in the controller class to handle specific requests (e.g., a method for handling product search requests).
- **Model Interaction:** Within the controller method, interact

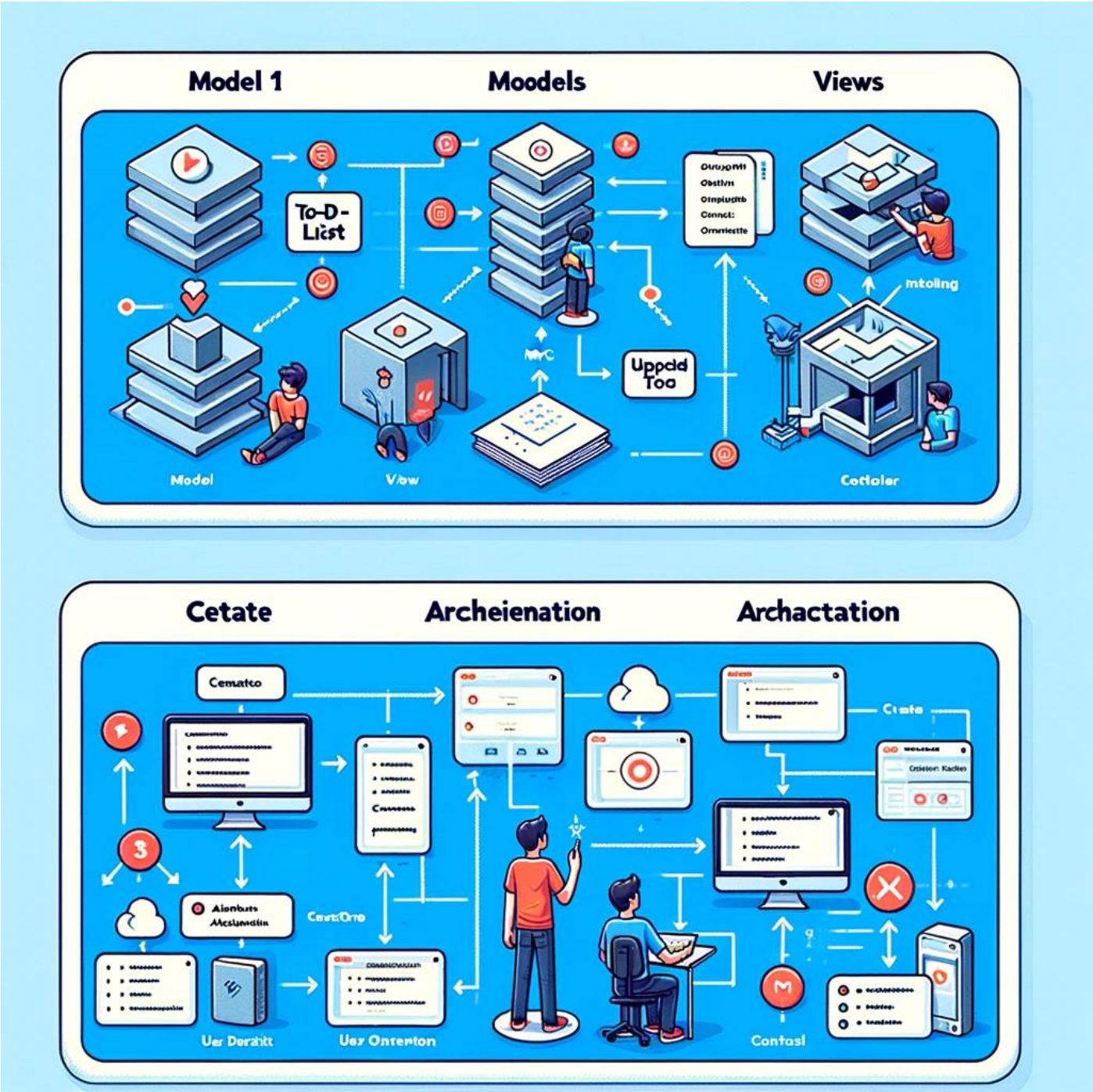
MVC Architecture in Cloud Applications

MVC architecture plays a crucial role in the development and management of cloudbased applications. Its separation of concerns facilitates the creation of scalable, maintainable, and efficient cloud services.

Key Benefits in the Cloud Context:

1. Scalability: MVC's modular nature allows cloud applications to scale more efficiently. Components can be scaled independently based on demand.
2. Maintenance: Simplifies updates and maintenance without disrupting the entire application, essential for the continuous delivery model in cloud computing.
3. **Development** Efficiency: Supports collaborative and parallel development, speeding up the deployment of new features or services in the cloud.
Application Areas:
Web Services and APIs: MVC is extensively used in developing RESTful web services and APIs that are consumed by various cloudbased applications.
Dynamic Web Applications: Cloud platforms host dynamic web applications that leverage MVC for responsive, usercentric design and functionality.
Enterprise Solutions: For enterprise cloud solutions requiring robust data handling, business logic, and user interface separation, MVC offers a structured approach to managing these complex requirements.

Lab Tasks:



Task 1: Create a Basic MVC Application

Example: Start by defining the model to represent a to-do item (e.g., TodoItem with properties like Id, Name, IsComplete). Create views to display the list of to-do items and a form to add new items. Implement a controller with actions to handle the display and creation of to-do items

Task:

Students will create a simple MVC application, such as a to-do list, to understand how models, views, and controllers interact.

Task 2: Implement CRUD Operations

Example: Expand the controller with actions for updating and deleting to-do items, such as Edit, Update, Delete. Use the model to interact with the database or in-memory data store for these operations. Update the views to include options for editing and deleting items.

Task:

Extend the basic application by implementing CRUD (Create, Read, Update, and Delete) operations for the to-do list items, demonstrating the handling of data in an MVC application.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 07

Objective: To implement preserving user state in distributed applications

Introduction:

Preserving user state in distributed applications is a critical aspect of modern web development, ensuring a seamless and continuous user experience across different sessions and application instances. As applications grow in complexity and scale across multiple servers or cloud environments, the challenge of maintaining user-specific data, such as login sessions, preferences, and application states, becomes increasingly significant. This practical explores the strategies and technologies used to address these challenges.

1. Understanding User State:

- User state refers to any data that reflects the user's interaction with an application across different components (services, microservices, UI).
- Common examples include:
  - Authentication details (e.g., user ID, logged-in status)
  - User preferences (e.g., language, theme)
  - Shopping cart contents
  - Application state (e.g., form data, progress within a workflow)

2. Choosing a Storage Mechanism:

The choice between client-side and server-side storage impacts performance, security, flexibility, and data persistence:

2.1 Client-Side Storage:

Pros:

- Faster access as data is readily available on the user's device.
- No reliance on server availability for basic functionality.

Cons:

- Security concerns: data is vulnerable to manipulation and theft.
- Limited persistence: data is lost once the user clears browser cache or cookies.
- No sharing across devices: the state is specific to the device used.

Suitable for:

- Non-sensitive data (e.g., language preference, theme)
- Temporary data used within a single session (e.g., form data before submission)

Common options:

- **Cookies:** Simple key-value pairs stored on the user's device with limited size and expiration times.
- **Local Storage:** Persistent key-value storage specific to a particular domain (origin).
- **Session Storage:** Similar to local storage but data is cleared upon browser tab/window closure.

2.2 Server-Side Storage:

Pros:

- More secure: data is stored on the server, mitigating client-side manipulation.
- Persistent: data can be accessed across user sessions and devices.
- Scalable: storage scales well with the application's needs.

Cons:

- Performance impact: additional network requests required to access and update data.
- Server dependency: application functionality relies on server availability.

Suitable for:

- Sensitive data (e.g., authentication details, user profile information)
- Application state requiring persistence across user sessions and devices (e.g., shopping cart contents)

Common options:

- **Databases:** (e.g., relational databases, NoSQL databases) offer structured storage for complex data and relationships.
- **Key-value stores:** (e..g., Redis, Memcached) provide fast access for frequently accessed data but lack complex data structures.

2.3 Session Management:

This approach leverages server-side storage but manages data specific to a user session:

- A unique session ID is assigned to the user upon login or initial interaction.
- This ID is typically stored in a client-side cookie and used to retrieve and associate user data stored on the server.
- Sessions have configurable expiration times to balance security and convenience.

3. Implementing State Management Pattern:

3.1 Client-Side:

- Use browser APIs for managing client-side storage (e.g., localStorage, sessionStorage).
- Consider libraries like Redux or MobX for complex state management within single-page applications.
- These libraries provide mechanisms for state updates, synchronization with server-side, and component re-rendering based on state changes.

3.2 Server-Side:

- Develop APIs to interact with the chosen server-side storage solution (database, key-value store).
- If using sessions, integrate a session management library to handle session creation, renewal, and invalidation.

4. Synchronizing User State:

4.1 Server-Side Updates:

- Upon user interaction (e.g., form submission, button click), send updates to the server using AJAX, RESTful APIs, or other data transfer methods.
- The server-side updates the chosen storage mechanism (database, key-value store) based on the received data.
- Session data may also be updated for session-based management.

4.2 Client-Side Updates:

- Fetch latest user data from the server on page load or specific events (e.g., login, session renewal).
- This can be achieved through API calls triggered by the client-side application.
- Update the client-side state with the retrieved data, potentially triggering UI updates to reflect the new state.

5. Additional Considerations:

5.1 Security:

- Always consider security implications when storing and handling user data.
- Implement encryption (e.g., HTTPS) for data transmission between client and server.
- Store sensitive data securely on the server

Lab Task:

Imagine you're building a basic online quiz:

Task 1: Keeping Track of User Progress

- **Scenario:** Users answer questions in a multi-step quiz.
- **Problem:** How can we keep track of which questions a user has answered and their current progress?
- **Hint:** Imagine each question as a colored piece of paper. We could have a basket representing the user's state (progress). When a user answers a question, we move the corresponding piece (question) into the basket.

Task 2: Remembering User Choices

- **Scenario:** The quiz allows users to choose their preferred language (English or Spanish).
- **Problem:** How can we remember the user's language preference throughout the quiz?
- **Hint:** Imagine a small sticky note stuck to the user's screen. They can write their preferred language on it (e.g., "English"). This sticky note represents the user's state (preference).

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 08

Objective: Spring Framework Practical: Building a Distributed Java Application

Description: In this lab, explore the fundamentals of the Spring framework, understand dependency injection, and build a distributed Java application using Spring.

1. Introduction to the Spring Framework:

The spring framework is a comprehensive framework for enterprise Java development. It provides a broad set of features, including Inversion of Control (IoC), Aspect-Oriented Programming (AOP), and a wide range of modules for various functionalities.

2. Key Concepts of the Spring Framework:

Dependency Injection (DI): Understand the concept of

Dependency Injection and how it promotes loose coupling.

Learn to use the '@Autowired' annotation for automatic dependency injection.

Inversion of Control (IoC): Explore IoC and how it

simplifies object creation and management.

Create a simple Spring application demonstrating IoC.

3. Building a Spring Application:

1. Step 1: Setting up the Development Environment:

- Install the latest version of the Spring framework or use Spring Boot.
- Configure your Integrated Development Environment (IDE) for Spring development.

Step 2: Creating a Simple Spring Application:

2.1 Define a Bean Class –'UserService':

```
package com.example.springdemo;

import org.springframework.stereotype.Service;

@Service

public class UserService {

   public String getUserInfo() {

      return "User information from UserService";

   }}
```

2.2 Configure the Spring IoC Container – 'AppConfig':

```
package com.example.springdemo;

import
org.springframework.context.annotation.Comp
onentScan;

import
org.springframework.context.annotation.Confi
guration;
```

```
@Configuration
@ComponentScan(basePackages =
"com.example.springdemo")
public class AppConfig {
   // No additional configuration needed for
this example
}
```

2.3 Develop a Main Application - 'MainApplication':

```
package com.example.springdemo;
import
org.springframework.context.annotation.Anno
tationConfigApplicationContext;
public class MainApplication {
   public static void main(String[] args) {
      // Load the Spring configuration
```

```
        AnnotationConfigApplicationContext
context =
            new
AnnotationConfigApplicationContext(AppConfi
g.class);
        // Get the bean from the Spring container
        UserService userService =
context.getBean(UserService.class);
        // Use the bean

System.out.println(userService.getUserInfo());
        // Close the context
        context.close();
    }}
```

4. Advanced Spring Features:

4.1 Aspect-Oriented Programming (AOP):

- Implement a simple aspect to log method calls or add additional behavior.

4.2 Data Access:

- Integrate Spring Data or JDBC to perform basic database operations.

4.3 RESTful Web Services:

- Create a RESTful endpoint using Spring MVC.

4.4 Explore Spring Boot:

- Research and experiment with Spring Boot for rapid application development.

4.5 Testing:

- Write unit tests for your Spring components using JUnit and Mockito.

Lab Task:

Now that you have a basic understanding of the Spring framework, enhance the application by:

1. Security:

Implement Spring Security to secure your application against unauthorized access.

2. Micro services Architecture:

Modify the application to work as a micro service and communicate with other micro services.

3. Messaging:

Integrate Spring Messaging for communication between different components.

4. Explore Spring Cloud:

Experiment with Spring Cloud for building distributed systems.

5. Error Handling:

- Implement global error handling mechanisms using Spring.
- By completing these tasks, you will deepen your knowledge of the Spring framework and gain practical experience in developing robust and scalable Java applications using Spring.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 09

Objective: To work with JDBC template of Spring framework

Description: The Spring framework's JDBC template offers a convenient and efficient way to interact with relational databases from your Java applications. It simplifies the complexities of raw JDBC by handling connection management, statement creation, and result set processing, allowing you to focus on writing cleaner and more maintainable code.

Here's a breakdown of key aspects:

What it does:

- Executes SQL queries (SELECT, INSERT, UPDATE, DELETE) and stored procedures.
- Maps result sets to Java objects or collections.
- Provides abstractions for prepared statements, parameter setting, and query execution.
- Handles database exceptions and translates them to Spring's own exception hierarchy.

1. Overview

Spring Data JDBC is a persistence framework that is not as complex as Spring Data JPA. It doesn't provide cache, lazy loading, write-behind, or many other features of JPA. Nevertheless, it has its own ORM and provides most of the features we're used with Spring Data JPA **like mapped entities, repositories, query annotations, and *JdbcTemplate***.

An important thing to keep in mind is that **Spring Data JDBC doesn't offer schema generation**. As a result, we are responsible for explicitly creating the schema.

2. Adding Spring Data JDBC to the Project

- Spring Data JDBC is available to Spring Boot applications with the JDBC dependency starter. **This dependency starter does not bring the database driver, though**. That decision must be taken by the developer. Let's add the dependency starter for Spring Data JPA:
- <dependency>
- <groupId>org.springframework.boot</groupId>
- <artifactId>spring-boot-starter-data-jdbc</artifactId>
- </dependency>Copy
- In this example, we're using the H2 database. As we mentioned early, Spring Data JDBC doesn't offer schema generation. In such a case, we can create a custom *schema.sql* file that will have the SQL DDL commands for creating the schema objects. Automatically, Spring Boot will pick this file and use it for creating database objects.

3. Adding Entities

- As with the other Spring Data projects, we use annotations to map POJOs with database tables. In **Spring Data JDBC, the entity is required to have an *@Id*.** Spring Data JDBC uses the *@Id* annotation to identify entities.
- Similar to Spring Data JPA, Spring Data JDBC uses, by default, a naming strategy that maps Java entities to relational database tables, and attributes to column names. By default, the Camel Case names of entities and attributes are mapped to snake case names of tables and columns, respectively. For example, a Java entity named *AddressBook* is mapped to a database table named *address_book*.

- Also, we can map entities and attributes with tables and columns explicitly by using the *@Table* and *@Column* annotations. For example, below we have defined the entity that we're going to use in this example:
- publicclassPerson {
- @Id
- privatelong id;
- private String firstName;
- private String lastName;
- // constructors, getters, setters
-)
- We don't need to use the annotation *@Table* or *@Column* in the *Person* class. The default naming strategy of Spring Data JDBC does all the mappings implicitly between the entity and the table.

4. Declaring JDBC Repositories

- Spring Data JDBC uses a syntax that is similar to Spring Data JPA. We can create a Spring Data JDBC repository by extending the *Repository*, *CrudRepository, or PagingAndSortingRepository* interface. By implementing *CrudRepository*, we receive the implementation of the most commonly used methods like *save*, *delete*, and *findById*, among others.
- Let's create a JDBC repository that we're going to use in our example:
- @Repository
- publicinterfacePersonRepositoryextendsCrudRepository<Person, Long> {
- }
- If we need to have pagination and sorting features, the best choice would be to extend the *PagingAndSortingRepository* interface.

5. Customizing JDBC Repositories

- Despite the *CrudRepository*'s built-in methods, we need to create our methods for specific cases.
- Now, let's customize our *PersonRepository* with a non-modifying query and a modifying query:

```
@Repository
publicinterfacePersonRepositoryextendsCrudRepository<Person, Long> {

  List<Person>findByFirstName(String firstName);

@Modifying
@Query("UPDATE person SET first_name = :name WHERE id = :id")
booleanupdateByFirstName(@Param("id") Long id, @Param("name") String name);
}
```

- Since version 2.0, Spring Data JDBC supports query methods. That is, if we name our query method including the keywords, for example, *findByFirstName,* Spring Data JDBC will generate the query object automatically.
- However, for the modifying query, we use the *@Modifying* annotation to annotate the query method that modifies the entity. Also, we decorate it with the *@Query* annotation.
- Inside the *@Query* annotation, we add our SQL command. **In Spring Data JDBC, we write queries in plain SQL.** We don't use any higher-level query language like JPQL. As a result, the application becomes tightly coupled with the database vendor.
- For this reason, it also becomes more difficult to change to a different database.
- One thing we need to keep in mind is that **Spring Data JDBC does not support the referencing of parameters with index numbers**. **We're able only to reference parameters by name**.

6. Populating the Database

- Finally, we need to populate the database with data that will serve for testing the Spring Data JDBC repository we created above. So, we're going to create a database seeder that will insert dummy data. Let's add the implementation of database seeder for thi

example:

```
@Component
publicclassDatabaseSeeder {

@Autowired
privateJdbcTemplatejdbcTemplate;
publicvoidinsertData() {
jdbcTemplate.execute("INSERT INTO Person(first_name,last_name) VALUES('Victor', 'Hugo')");
jdbcTemplate.execute("INSERT INTO Person(first_name,last_name) VALUES('Dante',
'Alighieri')");
jdbcTemplate.execute("INSERT INTO Person(first_name,last_name) VALUES('Stefan', 'Zweig')");
jdbcTemplate.execute("INSERT INTO Person(first_name,last_name) VALUES('Oscar', 'Wilde')");
   }
}
```

- As seen above, we're using Spring JDBC for executing the *INSERT* statements. In particular, Spring JDBC handles the connection with the database and lets us execute SQL commands using *JdbcTemplate*s. This solution is very flexible because we have complete control over the executed queries.

7. Conclusion

- To summarize, Spring Data JDBC offers a solution that is as simple as using Spring JDBC — there is no magic behind it. Nonetheless, it also offers a majority of features that we're accustomed to using Spring Data JPA.
- One of the biggest advantages of Spring Data JDBC is the improved performance when accessing the database as compared to Spring Data JPA. This is due to Spring Data

JDBC **communicating directly to the database**. **Spring Data JDBC doesn't contain most of the Spring Data magic when querying the database**.

- One of the biggest disadvantages when using Spring Data JDBC is the dependency on the database vendor. If we decide to change the database from MySQL to Oracle, **we might have to deal with problems that arise from databases having different dialects**.

Lab TASK:

1. **Query by ID:** Write a program that uses your PersonRepository to retrieve a Person object by its ID. Print the retrieved object's information.

2. **Insert New Person:** Create a new Person object with sample data and use the repository to save it to the database. Print the confirmation message or the saved object.

3. **Update First Name:** Find a specific Person by ID and update their first name using the repository's custom update method. Print the updated information retrieved again.

4. **Query by First Name:** Implement a custom query method that retrieves all Person objects with a specific first name. Print the list of retrieved objects.

5. **Delete Person:** Delete a specific Person object from the database using the repository. Print a confirmation message.

6. **Batch Insert:** Insert a list of multiple new Person objects into the database using batch processing techniques. Print the number of successfully inserted objects.

7. **Stored Procedure:** Implement a custom query method that calls a stored procedure defined in your database for a specific operation (e.g., calculating average age). Print the returned result.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 10

Objective: To work with cloud computing using Microsoft Azure

Description: Microsoft Azure is a comprehensive cloud computing platform offering a wide range of services, including:

Compute. These services enable a user to deploy and manage VMs, containers and batch jobs, as well as support remote application access. Compute resources created within the Azure cloud can be configured with either public IP addresses or private IP addresses, depending on whether the resource needs to be accessible to the outside world.

Mobile. These products help developers build cloud applications for mobile devices, providing notification services, support for back-end tasks, tools for building application program interfaces (APIs) and the ability to couple geospatial context with data.

Web. These services support the development and deployment of web applications. They also offer features for search, content delivery, API management, notification and reporting.

Storage. This category of services provides scalable cloud storage for structured and unstructured data. It also supports big data projects, persistent storage and archival storage.

Analytics. These services provide distributed analytics and storage, as well as features for real-time analytics, big data analytics, data lakes, machine learning, business intelligence, internet of things (IoT) data streams and data warehousing.

Networking. This group includes virtual networks, dedicated connections and gateways, as well as services for traffic management and diagnostics, load balancing, DNS hosting and network protection against distributed denial-of-service (DDoS) attacks.

Media and content delivery network (CDN). These CDN services include on-demand streaming, digital rights protection, encoding, and media playback and indexing.

Integration. These are services for server backup, site recovery and connecting private and public clouds.

Identity. These offerings ensure only authorized users can access Azure services and help protect encryption keys and other sensitive information in the cloud. Services include support for Azure Active Directory and multifactor authentication.

IoT. These services help users capture, monitor and analyze IoT data from sensors and other devices. Services include notifications, analytics, monitoring and support for coding and execution.

DevOps. This group provides project and collaboration tools, such as Azure DevOps -- formerly Visual Studio Team Services -- that facilitate DevOps software development processes. It also offers features for application diagnostics, DevOps tool integrations and test labs for build tests and experimentation.

Development. These services help application developers share code, test applications and track potential issues. Azure supports a range of application programming languages, including JavaScript, Python, .NET and Node.js. Tools in this category also include support for Azure DevOps, software development kits (SDKs) and blockchain.

Security. These products provide capabilities to identify and respond to cloud security threats, as well as manage encryption keys and other sensitive assets.

AI and machine learning. This is a wide range of services that a developer can use to infuse AI, machine learning and cognitive computing capabilities into applications and data sets.

Containers. These services help an enterprise create, register, orchestrate and manage huge volumes of containers in the Azure cloud, using common container platforms such as Docker and orchestration platforms including Kubernetes.

Databases. This category includes database as a service (DBaaS) offerings for SQL and NoSQL, as well as other database instances -- such as Azure Cosmos DB and Azure Database for PostgreSQL. It also includes Azure SQL Data Warehouse support, caching, and hybrid database integration and migration features. Azure SQL is the platform's flagship database service. It is a relational database that provides SQL functionality without the need for deploying a SQL server.

How does Microsoft Azure Cloud work?

Azure is an enormous assortment of systems administration equipment and servers that run a perplexing arrangement of uses broadly circulated in an association. These applications make the arrangement and activity of virtualized programming and equipment work in a state of harmony on the servers. Microsoft Azure is a cloud computing stage that works by giving a variety of services and assets over the web. It works on a worldwide organization of data centers and permits organizations and people to construct, convey, and oversee applications and administrations without putting resources into and keeping up with their own actual foundation. This is an outline of the way **Microsoft Azure Cloud** works:

- **Data Centers**

Azure works in a worldwide organization of data centers decisively dispersed across the world. These data centers are outfitted with cutting-edge equipment and a systems administration framework. They are liable for facilitating the physical and virtual assets that makeup Azure's cloud administrations (services).

- **Virtualization**

At the core of Azure's activity is virtualization innovation. This innovation empowers the reflection of actual equipment assets, like servers, stockpiling, and systems administration, into virtual occurrences. These occurrences are known as Virtual Machines (VMs). Azure proposes a hypervisor to oversee and designate these VMs on the fundamental actual equipment. This virtualization layer is urgent for Azure's capacity to give versatile and adaptable **cloud services**.

In outline, **Microsoft Azure Cloud** works by utilizing a worldwide organization of data centers, virtualization innovation, and a huge swath of administrations to give versatile, adaptable, and secure distributed computing arrangements. Azure Resource Manager, network choices, character and access the executives, security highlights, checking devices, and computerization capacities structure the underpinning of Azure's usefulness, making it a flexible stage for different distributed computing needs. Clients can convey and deal with their assets and applications while profiting from Microsoft's skill in dealing with the basic framework.

Step by Step Guide to Getting Started with Microsoft Azure

Getting started with Microsoft Azure can appear to be overwhelming, yet with the right methodology, you can explore the stage. Here is a step-by-step guide to assist you with starting your excursion with Microsoft Azure:

1: Signing in for Azure

- Log into the website of Azure: Click on the link present in the portal of Azure as https://portal.azure.com/.
- Create an account with Azure: Firstly, it is essential to make an account on Azure. Follow the on-screen prompts to give essential data, including installment subtleties. Azure frequently offers a free preliminary with credits, which you can use to investigate the platform.
- Identity verification: Microsoft might expect you to confirm your identity through a call or SMS message.

2: Exploring the portal of Azure

- Log in to the portal of Azure: Use the credentials of the Azure account to sign in to the portal of Azure.
- Familiarizing with the dashboard: Now take some time to explore the features and layouts present on the platform of Azure. You'll find different administrations and assets coordinated into classes on the left sidebar.

3: Creating the first resource

- **Creating a resource:** Click the "+ Create a resource" button on the portal of the Azure landing page. This will open the Azure Commercial Center, where you can choose from a large number of resources as well as services
- **Selecting the service:** You can begin with a normal asset like a Virtual Machine, Web Application, or an Information base. Click on the selected service, to begin with the process of configuration.
- **Configuring the resource:** Follow the prompts to design your asset. This might incorporate picking a district, indicating asset subtleties, and setting up validation and options for networking.
- **Creating and Reviewing:** Subsequent to configuring the resource, review the settings, and click the "Create tab" to arrange it. Azure will convey your asset in light of your determinations.

4: Exploring documentation and tutorials

- **Documentation of Azure:** Visit the Azure documentation at https://docs.microsoft.com/en-us/Azure/. Here, you'll track down nitty gritty aides, instructional exercises, and documentation for different services of Azure.
- **Azure Quickstart Templates:** Investigate the Azure Quickstart Formats at https://azure.microsoft.com/en-us/assets/layouts/. These formats give pre-designed solutions that you can convey with only a couple of clicks.

5: Learning the fundamentals of Azure

- **Learning Microsoft:** Microsoft offers a free stage called Microsoft Learn at https://learn.microsoft.com/. It gives intuitive, active modules and learning ways explicitly intended to assist you with understanding the fundamentals of Azure.
- **Certifications:** In the event that you're keen on validating the information of Azure, consider pursuing a certification of Azure, for example, the Microsoft Affirmed: Azure Essentials or other job-based certificates.

6: Learning and Optimizing continuously

- **Stay Updated**: Azure tends to have innovative and new features that are evolving constantly. It is essential to adapt to the new changes and take the full benefit of the services being offered by Azure.
- **Optimizing the usage of data**: Consistently reviewing and enhancing the resources of Azure and spending to guarantee cost-adequacy and proficiency.
- **Getting everything started with Microsoft Azure:** It is an excursion that can prompt energizing opportunities for your tasks and business. By following these

means and ceaselessly extending your insight, you'll be well headed to becoming capable of the variety of services of Azure.

Selecting a Microsoft Cloud Solution Provider

Choosing a **Microsoft Cloud Solution Provider (CSP)** is a significant choice for associations hoping to **use Cloud computing services** successfully. A **Cloud Solution Provider** is an accomplice authorized by Microsoft to sell and oversee Microsoft cloud contributions, including Microsoft 365, Dynamics 365, and Azure in the interest of clients.

Here are key contemplations while selecting a Microsoft Cloud Solution Provider.

1. Most importantly, evaluating your association's particular necessities and objectives is fundamental. Obviously characterize why you are **selecting a Cloud Solution Provider**.
2. Understanding your spending plan limitations and setting practical assumptions about the costs of service is significant. Investigating potential **Microsoft Cloud Solution Providers** is the following basic step.
3. Start by investigating Microsoft's true Accomplice Catalog to distinguish approved CSPs in your district. Understanding audits and tributes from past clients can offer significant experiences in the nature of administrations. Look for proposals from partners or industry companions to additionally approve a reputation **Cloud Solution Providers**.
4. Assessing the contributions of potential Cloud Solution Providers is fundamental which guarantees they give the particular **Microsoft cloud services** you require and look at valuing structures, charging choices, and authoritative terms.
5. Furthermore, evaluate their help capacities, including time to give response, accessibility, and Service Level Agreements (SLAs). Check their obligation to security and consistency principles applicable to your industry.
6. Conducting interviews with shortlisted Cloud solution providers to acquire further bits of knowledge about their experience, adaptability choices, relocation cycles, and correspondence channels.

Lab Task:

1. **Setting Up Your Azure Environment:**
   - Task 1.1: Sign up for a free Azure account.
   - Task 1.2: Spend 5 minutes exploring the main sections of the Azure portal.
   - Task 1.3: Download and install the Azure CLI.
   - Task 1.4: Use the az account show command to verify your connection.
2. **Exploring Azure Services**:
   - Task 2.1: Choose one service category (e.g., compute) and browse 3-4 services within it.
   - Task 2.2: For each service, briefly summarize its main purpose in 1-2 sentences.
   - Task 2.3: Pick one service and find one real-world use case example online.
3. **Creating Your First Azure Resource:**
   - Task 3.1: Create a basic Virtual Machine named "MyFirstVM" with the smallest size option.
   - Task 3.2: Choose a resource group named "MyResourceGroup" and the closest location to your university.
   - Task 3.3: Configure basic security by allowing access from your local machine.
   - Task 3.4: Connect to your VM using the Remote Desktop Protocol (RDP) and verify its functionality.
4. **Deploying a Simple Web Application**:
   - Task 4.1: Create a simple HTML page with "Hello, World!" and your university name.
   - Task 4.2: Explore the "App Service" section in the Azure portal.
   - Task 4.3: Choose a free pricing tier and deploy your web page as an App Service application.
   - Task 4.4: Access your deployed application through the provided URL and see your message.
5. **Experimenting with Azure Functions:**
   - Task 5.1: Learn about Azure Functions by reading a short introductory article.
   - Task 5.2: Create a simple HTTP-triggered function that returns "Hello from Azure Functions!"
   - Task 5.3: Deploy your function and test it by sending an HTTP request using a tool like Postman.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 11

Objective: **To work with Amazon Web Services (AWS)**

Description: In this lab, students will transition from local development to deploying a distributed application on Amazon Web Services (AWS). They will gain hands-on experience in setting up AWS infrastructure, deploying applications on AWS Elastic Compute Cloud (EC2) instances, configuring networking to enable communication between distributed components, and monitoring application performance. Through this practical exercise, students will understand the fundamentals of cloud computing and how to leverage AWS services for building scalable and resilient distributed systems.

Introduction to AWS:

Amazon Web Services (AWS) is a leading cloud computing platform that offers a wide range of services, including storage, computing power, databases, machine learning services, and more. AWS facilitates businesses and individual users with hosting applications, securely storing data, and utilizing various tools and services to improve management flexibility for IT resources.

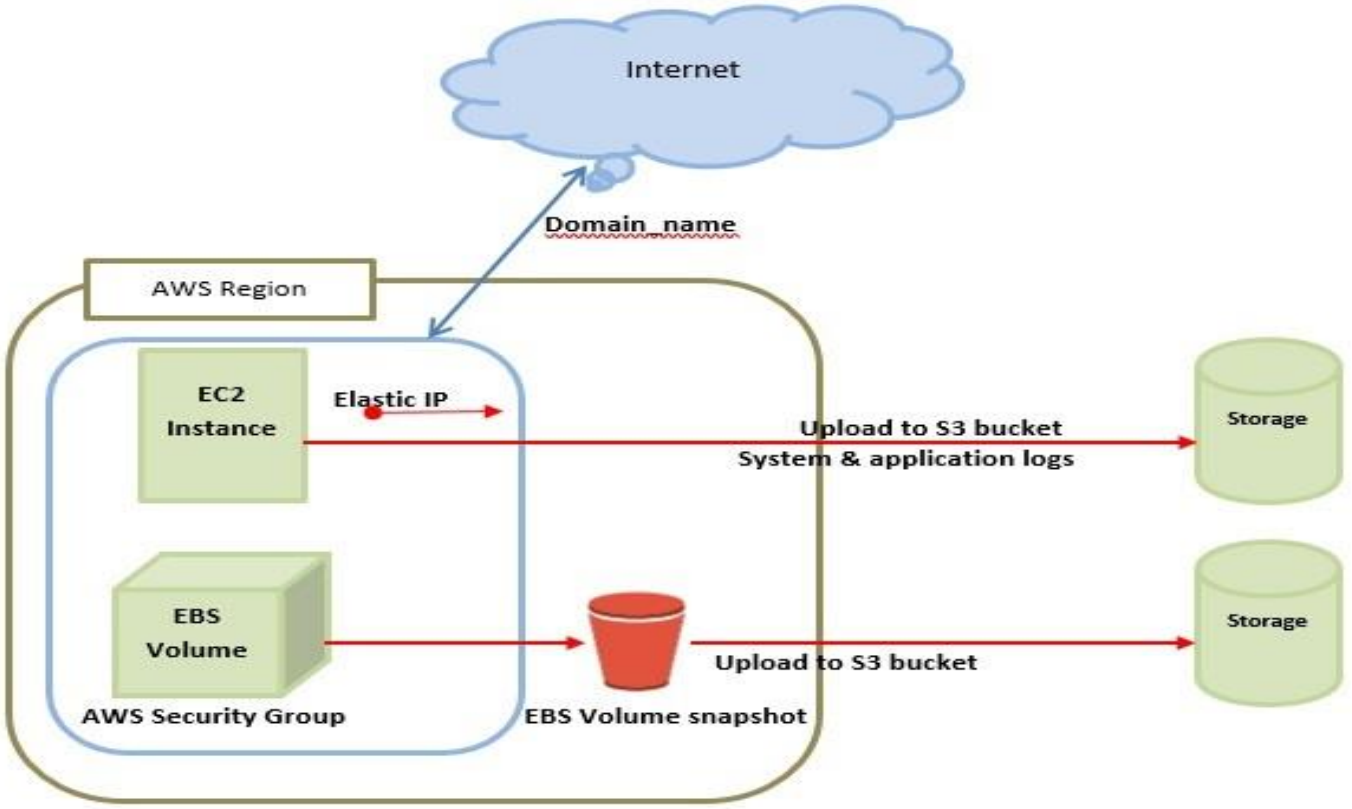
How AWS Works:

AWS establishes its network infrastructure by establishing data centers in different regions worldwide. It provides services such as Amazon EC2 and Amazon S3 for compute and storage, respectively, with elastic scaling capabilities. AWS also offers services like Auto Scaling and Elastic Load Balancing for dynamic scaling of applications.

AWS Fundamentals:

Understanding key concepts such as Regions, Availability Zones (AZ), and Global Network Infrastructure is crucial for maintaining reliable and scalable applications on AWS. Regions are geographical areas with data centers, while AZs are isolated locations within regions to enhance fault tolerance and disaster recovery management. AWS maintains a global network infrastructure to ensure reliability and scalability of services.

AWS Basic Architecture:



Top AWS Services:

- **Amazon EC2 (Elastic Compute Cloud):** Provides scalable computing power via the cloud.
- **Amazon S3 (Simple Storage Service):** Offers scalable object storage with high durability.
- **AWS Lambda:** Enables serverless computing by running code in response to events.
- **Amazon RDS (Relational Database Service):** Simplifies database management in the cloud. Amazon VPC (Virtual Private Cloud): Creates isolated networks within AWS for safe and adaptable resource configurations.

Advantages of Amazon Web Services:

- Easily scalable resources with a pay-as-you-go pricing model.
- Highly reliable and secure infrastructure with multiple data centers.
- Wide range of services and tools for flexibility in application development.

- Pay-as-you-go pricing model reduces upfront costs and long-term commitments.

Disadvantages of Amazon Web Services:

- Complexity can be challenging for new users.
- Costs can escalate, especially for high-traffic applications.
- Securing resources can be complex, requiring additional measures.
- Limited control over certain aspects of the infrastructure.

Applications of AWS:

- **Netflix:** Utilizes AWS for seamless content delivery to millions of users globally.
- **Airbnb:** Manages workloads and infrastructure for its virtual marketplace and lodging offerings.
- **NASA's Jet Propulsion Laboratory:** Analyzes large-scale data for scientific research missions.
- **Capital One:** Ensures security and compliance while delivering innovative banking services.

Practical Implementation:

Now, let's proceed with the practical implementation of deploying a simple distributed application on AWS.

Step 1: Setting up AWS Infrastructure

- Sign in to the AWS Management Console.
- Navigate to the EC2 service and launch EC2 instances in your preferred region.
- Configure security groups to allow inbound traffic on necessary ports.

Step 2: Deploying the Distributed Application

- Prepare your application code and dependencies for deployment.
- Connect to your EC2 instances using SSH or Remote Desktop Protocol (RDP).
- Install necessary software and deploy your application code.
- Configure load balancing and auto-scaling if required for scalability.

Step 3: Configuring Networking

- Set up a Virtual Private Cloud (VPC) to isolate your network resources.
- Create subnets within the VPC and configure routing tables.
- Configure internet gateways and NAT gateways for internet access.
- Configure security groups to control inbound and outbound traffic.

Step 4: Testing and Monitoring

- Test your distributed application to ensure proper functionality.
- Set up monitoring and logging using AWS CloudWatch.
- Monitor resource utilization and performance metrics.

Conclusion:

In this practical, students have learned how to deploy a simple distributed application on Amazon Web Services (AWS). They have gained hands-on experience in setting up AWS infrastructure, deploying applications on EC2 instances, configuring networking, and monitoring application performance. By completing this practical, students are better equipped to leverage AWS services for building scalable and resilient distributed systems.

Lab Task:

1. Your task is to enhance the deployed distributed application by implementing additional features or functionalities. You can explore integrating other AWS services such as Amazon DynamoDB for data storage, Amazon SQS for message queuing, or AWS Lambda for serverless computing to improve the application's scalability, reliability, and performance.
2. Additionally, experiment with advanced AWS configurations such as load balancing, auto-scaling, and fault tolerance mechanisms to optimize the application's architecture for real-world deployment scenarios. Test the application thoroughly to ensure that it meets performance and reliability requirements under varying loads and conditions.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

Practical 12

Objective: **Develop Distributed Applications using Amazon Web Services (AWS)**

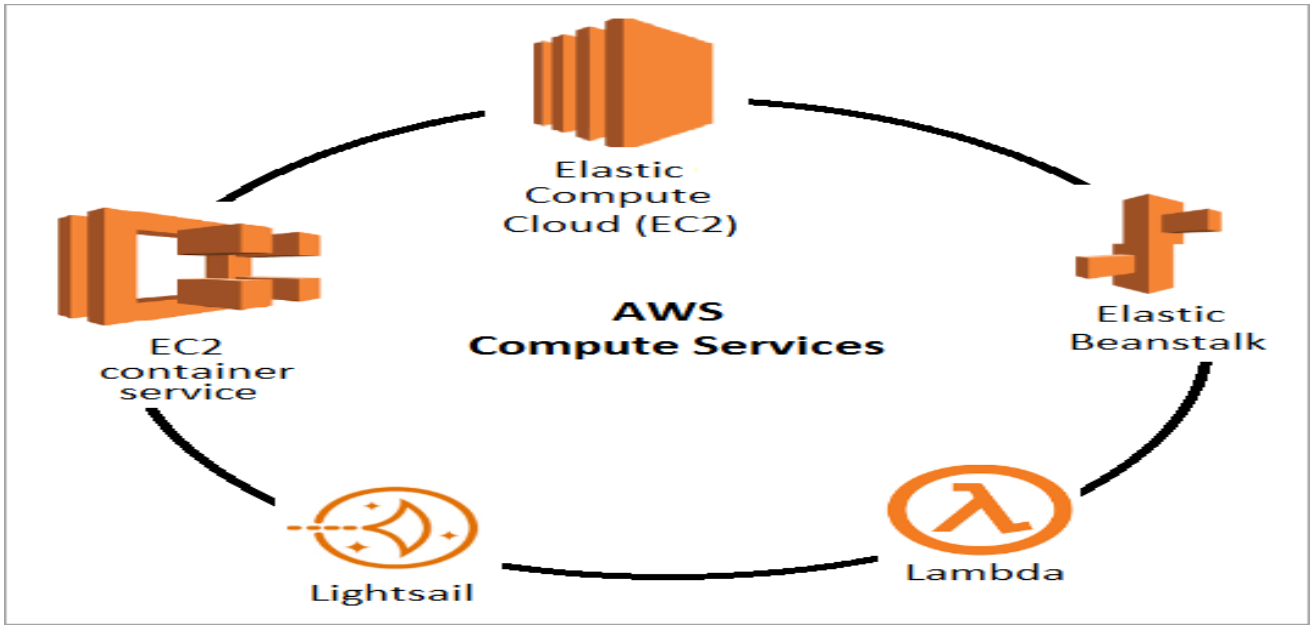
Description: In this lab, students will learn how to develop a distributed application using Amazon Web Services (AWS). Through step-by-step guidance, students will gain hands-on experience in setting up AWS infrastructure, deploying application components, configuring networking, and monitoring application performance.

Distributed Application:
A distributed application refers to a software application that consists of multiple components or services running on different machines or servers. Amazon Web Services (AWS) provides various services and tools that facilitate the development, deployment, and management of distributed applications.

How can you architect a Distributed Application using AWS?

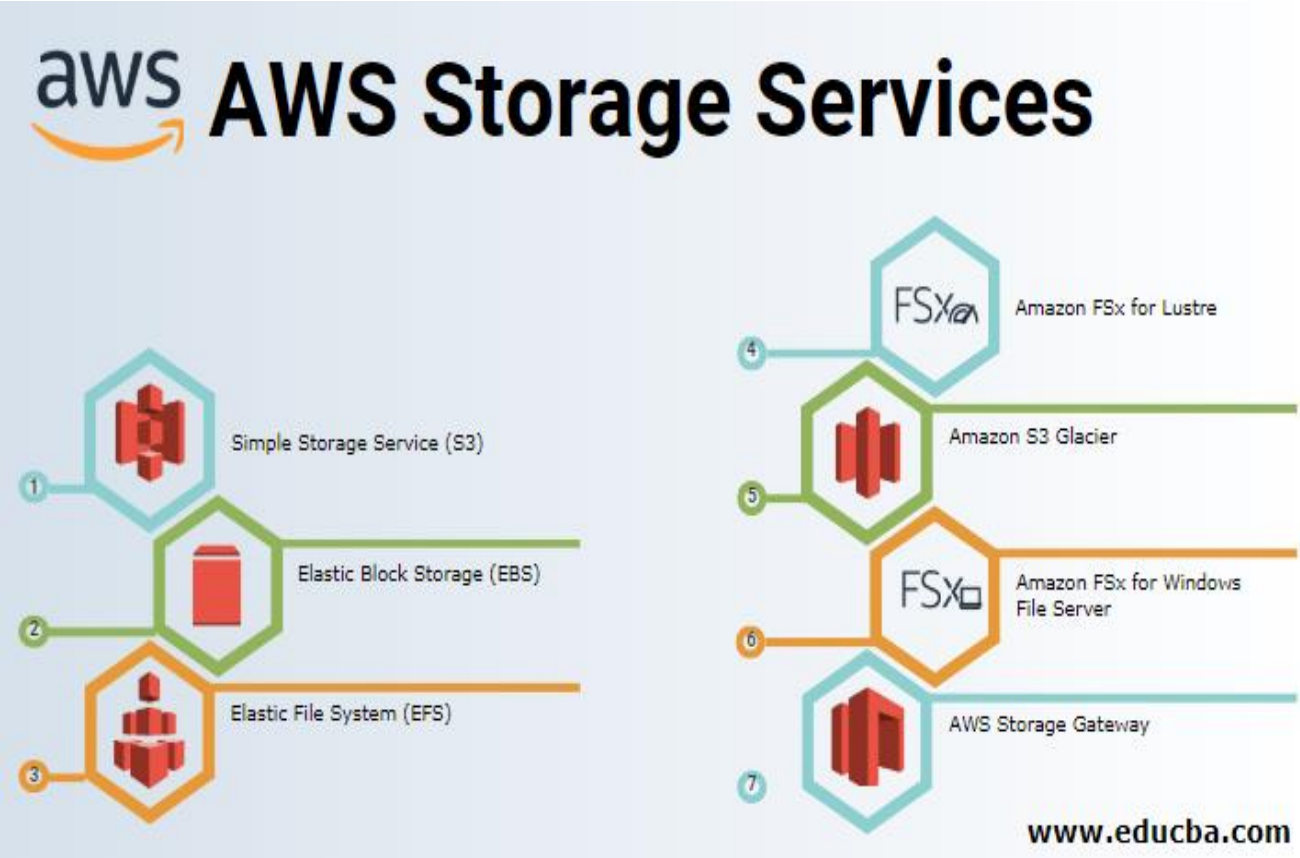
1. **Compute Services**:

- **Amazon EC2 (Elastic Compute Cloud)**: EC2 allows you to launch virtual servers (instances) on the cloud. You can distribute your application's components across multiple EC2 instances to handle different tasks or services.
- **AWS Lambda**: Lambda enables you to run code without provisioning or managing servers. You can use Lambda for serverless computing, where individual functions within your application can be triggered by events and scaled automatically.



2. **Storage Services**:

- **Amazon S3 (Simple Storage Service)**: S3 provides object storage that you can use to store and retrieve any amount of data. You can store static assets, user uploads, and other data required by your distributed application.
- **Amazon EBS (Elastic Block Store)**: EBS offers block-level storage volumes for use with EC2 instances. You can attach EBS volumes to your EC2 instances to provide persistent storage for your application data.



3. **Database Services**:

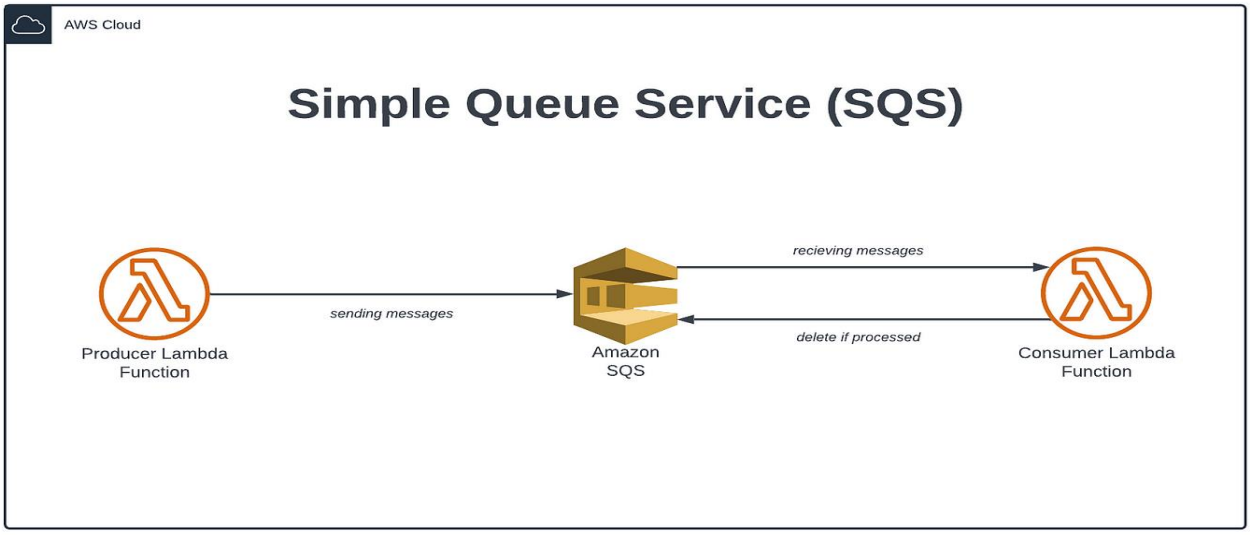
- **Amazon RDS (Relational Database Service)**: RDS provides managed relational database solutions such as MySQL, PostgreSQL, Oracle, and SQL Server. You can use RDS to store structured data for your application.
- **Amazon DynamoDB**: DynamoDB is a fully managed NoSQL database service. It offers low-latency, high-performance database capabilities, making it suitable for handling large-scale, distributed applications.

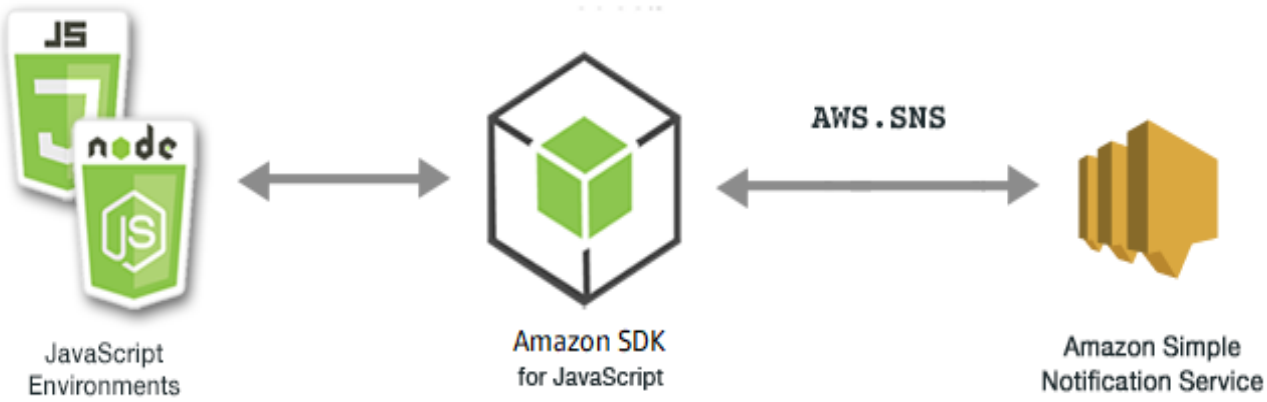
4. **Networking**:

- **Amazon VPC (Virtual Private Cloud)**: VPC enables you to provision a logically isolated section of the AWS Cloud where you can launch AWS resources. You can configure network settings, subnets, route tables, and gateways to control the network environment for your distributed application.
- **Amazon Route 53**: Route 53 is a scalable DNS (Domain Name System) web service. You can use Route 53 to route traffic to your application endpoints, such as EC2 instances or load balancers, and to register domain names for your application.

5. **Messaging and Queuing**:

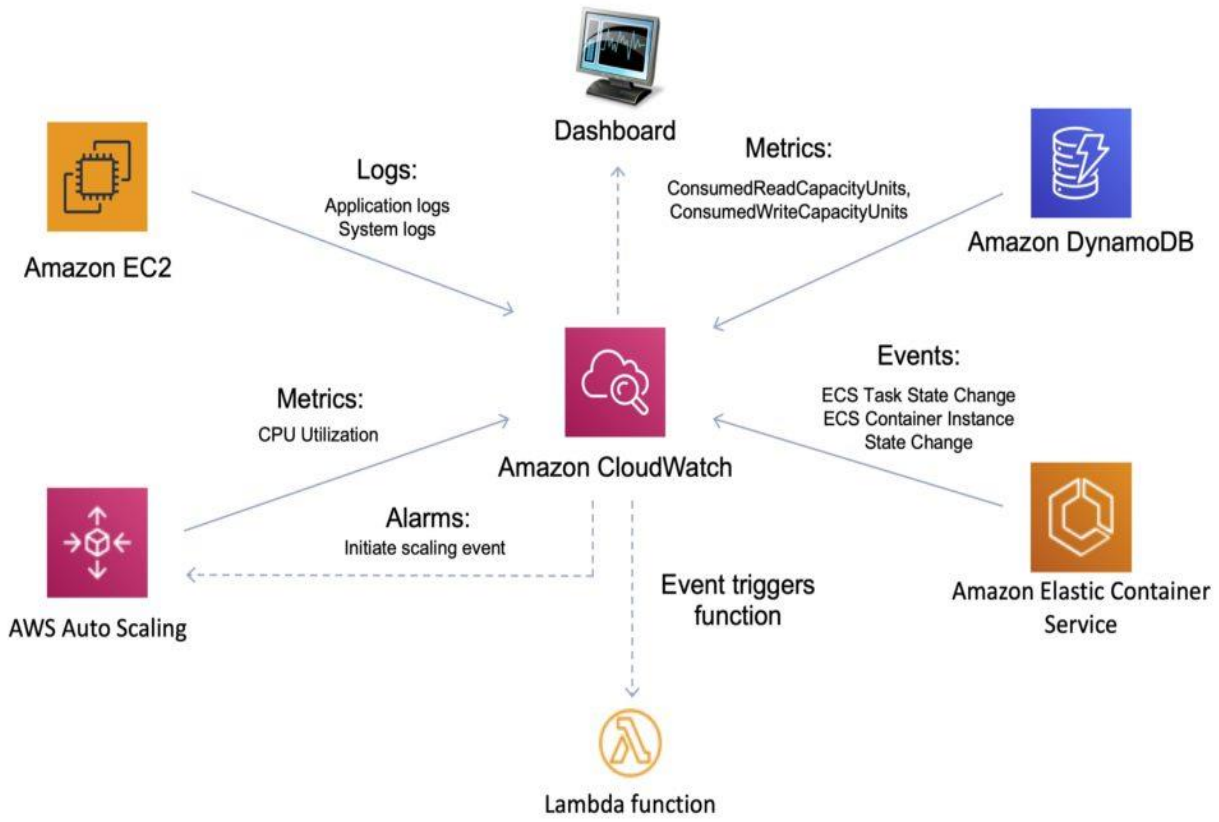
- **Amazon SQS (Simple Queue Service)**: SQS offers fully managed message queues for storing messages as they travel between distributed components of your application. It helps decouple application components and provides fault tolerance.
- **Amazon SNS (Simple Notification Service)**: SNS is a fully managed messaging service for publishing messages from your application to distributed components. It supports both push and pub/sub messaging models.





6. **Monitoring and Management**:

- **Amazon CloudWatch**: CloudWatch provides monitoring for AWS resources and applications running on AWS. You can collect and track metrics, monitor log files, set alarms, and automatically react to changes in your AWS resources.



When architecting a distributed application on AWS, you'll typically design for scalability, reliability, and fault tolerance by leveraging these AWS services in combination with best practices such as load balancing, auto-scaling, and redundancy across multiple availability zones. Additionally, you may use AWS CloudFormation or AWS CDK (Cloud Development Kit) to automate the deployment and management of your infrastructure as code.

Practical Implementation:

Let's proceed with the practical implementation of creating a distributed application using AWS:

Step 1: Setting up AWS Infrastructure

- Sign in to the AWS Management Console.
- Navigate to the EC2 service and launch EC2 instances in your preferred region.
- Configure security groups and key pairs as needed.

Step 2: Deploying Application Components

- Prepare application components, such as web servers, databases, and micro services.
- Deploy application components on EC2 instances or container services like Amazon ECS or AWS Fargate.
- Configure auto-scaling groups and load balancers for scalability.

Step 3: Configuring Networking

- Set up a Virtual Private Cloud (VPC) with public and private subnets.
- Configure routing tables and internet gateways for inbound and outbound traffic.
- Use security groups to control traffic flow.

Step 4: Monitoring Application Performance

- Set up monitoring using AWS CloudWatch.
- Create metrics and alarms to monitor resource utilization and performance.
- Use AWS X-Ray for distributed tracing and performance analysis.

Conclusion:

In this lab, students have learned how to create a distributed application using Amazon Web Services (AWS). By leveraging AWS services for infrastructure provisioning, deployment, networking, and monitoring, students have gained practical experience in building scalable and reliable distributed systems.

Lab Task:

1. Your task is to extend the distributed application by implementing additional features or functionalities. Consider integrating other AWS services such as Amazon DynamoDB for NoSQL data storage, Amazon SQS for message queuing, or AWS Lambda for serverless computing to enhance the application's functionality and performance.
2. Additionally, experiments with advanced AWS configurations such as cross-region replication, multi-AZ deployment, and hybrid cloud connectivity to optimize the application's architecture for resilience and fault tolerance.

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	Components	Level of Achievement			Marks Achieved
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

PRACTICAL NO: 13

Objective: To become familiar with Amazon's AWS Identity Management and Security in the Cloud

Description:

AWS Identity and Access Management (IAM) is a service that helps you securely manages who and what can access AWS resources. IAM lets you:

1. IAM - Fine-Grained Access Control:

Users and Groups:

- Create IAM users for individuals requiring access to your AWS resources.
- Organize users into IAM groups based on shared permission needs, simplifying permission management.
- Utilize IAM roles to grant temporary access to applications or services, eliminating the need for long-lived credentials.

Permissions:

- Define granular permissions using IAM policies, JSON documents specifying allowed actions on specific AWS resources (e.g., launching EC2 instances, accessing S3 buckets).
- Attach policies to users, groups, or roles to control their access and adhere to the principle of least privilege, granting only the minimum permissions necessary.

Security Measures:

- Enforce multi-factor authentication (MFA), requiring a second authentication factor (e.g., a code from your phone) for login attempts, significantly reducing the risk of unauthorized access.
- Enable IAM access logging to track user activity and identify potential security concerns by recording who accessed what resources and when.
- Leverage AWS Security Token Service (STS) to issue temporary credentials with limited durations and permissions, reducing the risk associated with compromised credentials.

2. Enhanced Security in the Cloud:

While IAM focuses on access control, AWS offers a comprehensive security suite for additional layers of protection:

- **Encryption:** Protect data at rest (e.g., in S3) and in transit (e.g., during network transfer) using industry-standard encryption algorithms and services like AWS Key Management Service (KMS) to manage encryption keys securely.
- **Network and Application Protection:**
  - Deploy AWS WAF (Web Application Firewall) to shield web applications from common attacks like SQL injection and cross-site scripting.
  - Implement Amazon GuardDuty for continuous threat detection and monitoring of your resources, identifying and responding to potential security issues.
- **Compliance and Threat Detection:**
  - Utilize AWS Security Hub to centralize security findings across your AWS accounts, providing a consolidated view of security posture and facilitating comprehensive risk management.
  - Leverage Amazon Inspector to scan your resources for vulnerabilities and security misconfigurations, proactively identifying potential security weaknesses.

3. Continuous Learning and Practice:

- Experiment with the AWS Free Tier: Utilize the free tier to explore IAM, security services, and deploy sample applications in a safe environment to gain practical experience.
- Enroll in AWS Skill Builder: Access free online courses and labs to solidify your understanding through guided practice and hands-on activities.

<u>How it works:</u>

1. Authentication:

- Who/What Requests Access?
- Users (individuals) authenticate using their username and password or access keys.
- Applications can utilize temporary credentials issued by IAM roles.
- Federated users authenticate through an external identity provider linked to your AWS account.

2. Authorization:

- Is Access Allowed?
- IAM checks the requesting entity's credentials (e.g., username/password, access key) and verifies their validity.
- It then evaluates attached IAM policies associated with the user, group, or role to determine if the requested action on the specific resource is permitted.
- This process ensures only authorized entities with the appropriate permissions can access your AWS resources.

3. Centralized Management:

- **Single Point of Control:**
  - The IAM console or API allows you to create and manage users, groups, roles, and policies from a central location.
  - This simplifies administration and ensures consistent access control across your entire AWS environment.

4. Fine-Grained Permissions:

- **Granular Control:**
  - IAM policies are defined using JSON documents and specify:
  - The specific actions allowed (e.g., create EC2 instances, list S3 buckets).
  - The resources these actions can be performed on (e.g., specific EC2 instance types, designated S3 buckets).
  - Any conditions that must be met for the action to be allowed (e.g., time of day restrictions, specific IP address origins).
  - This granular control allows you to grant users only the minimum permissions necessary for their tasks, adhering to the principle of least privilege.

5. Access Analysis:

- **Refine and Optimize:**

- IAM access analyzer allows you to evaluate existing policies and identify potential over-permissions.
- This helps you refine policies to grant only the necessary permissions, reducing the attack surface and improving security posture.

Lab Tasks:

1. **Create an IAM user:** Practice creating an IAM user with specific permissions for a particular task.
2. **Set IAM policies:** Grant the created user the necessary permissions to perform the designated task.
3. **Configure MFA:** Implement multi-factor authentication for the user account for added security.
4. **Review IAM policies:** Analyze existing IAM policies and identify any potential over-permission.

Qaid-e-Awam University of Engineering Science and Technology Nawasbshah

Department of Software Engineering

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	**Components**	**Level of Achievement**			**Marks Achieved**
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date: _____**

PRACTICAL NO: 14

Objective: To make Amazon's Restful Web Services

Description:

Amazon's RESTful Web Services (AWS) provide developers with a set of flexible and scalable cloud computing services designed to facilitate building and deploying applications and services through the internet. REST (Representational State Transfer) is an architectural style that uses standard HTTP methods such as GET, POST, PUT, DELETE to manipulate resources. AWS offers a variety of services that adhere to RESTful principles, allowing developers to interact with cloud resources programmatically.

AWS RESTful Web Services include but are not limited to:

1. **Amazon S3 (Simple Storage Service):** A scalable object storage service designed to store and retrieve any amount of data from anywhere on the web. Developers can use RESTful APIs to manage buckets (containers for storing objects) and objects (files) stored within them.
2. **Amazon EC2 (Elastic Compute Cloud):** A web service that provides resizable compute capacity in the cloud. Through RESTful APIs, developers can launch and manage virtual servers (EC2 instances) on-demand, allowing for flexible scaling of computing resources.
3. **Amazon API Gateway:** A fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. It enables the creation of RESTful APIs for applications running on AWS Lambda, EC2, or other publicly hosted web services.
4. **Amazon DynamoDB**: A fully managed NoSQL database service designed for applications requiring single-digit millisecond latency at any scale. RESTful APIs allow developers to interact with DynamoDB tables, querying and manipulating data using standard HTTP methods.
5. **Amazon RDS (Relational Database Service):** A managed database service that supports multiple database engines such as MySQL, PostgreSQL, SQL Server, and Oracle. Developers can use RESTful APIs to provision, manage, and scale relational databases in the cloud.

Steps for Creating Amazon RESTful Web Services:

1. **Sign in to the AWS Management Console:** Access the AWS Management Console using your AWS account credentials.

2. **Navigate to the desired service:** Choose the AWS service you want to use to create RESTful web services (e.g., Amazon S3, Amazon API Gateway) from the services menu.
3. **Create or configure resources:** Depending on the selected service, create or configure the necessary resources such as buckets (Amazon S3), EC2 instances (Amazon EC2), APIs (Amazon API Gateway), tables (Amazon DynamoDB), or databases (Amazon RDS).
4. **Generate API keys and endpoints:** If required, generate API keys and endpoints for your RESTful web services to securely access and interact with them programmatically.
5. **Define API methods and resources:** For services like Amazon API Gateway, define API methods (e.g., GET, POST, PUT, DELETE) and resources (e.g., /items, /users) to specify the functionality and structure of your RESTful API.
6. **Deploy the API:** Deploy your RESTful API to make it accessible over the internet. This typically involves configuring deployment settings and specifying the stage environment (e.g., development, production).
7. **Test the API:** Test your RESTful web services to ensure they function as expected. Use tools like cURL, Postman, or AWS SDKs to send requests and verify responses.
8. **Monitor and manage:** Monitor the performance and usage of your RESTful web services using AWS CloudWatch and other monitoring tools. Manage access control, authentication, and authorization settings as needed.

By following these steps, you can create and deploy RESTful web services on Amazon Web Services, leveraging the scalability, reliability, and flexibility of the AWS cloud infrastructure.

Lab Task:

1. Setup AWS Account:
   - If you haven't already, sign up for an AWS account or use an existing one.
2. Access Amazon API Gateway:
   - Log in to the AWS Management Console.
   - Navigate to the Amazon API Gateway service.
3. Create a New API:
   - Click on "Create API."
   - Choose the "REST API" option.
   - Select "New API" and give your API a name and description.

4. Define Resources and Methods:
   - Under your API, click on "Actions" and select "Create Resource."
   - Define a resource path (e.g., /items) and create a corresponding method (e.g., GET).
   - Configure the method integration with a backend service (e.g., AWS Lambda function, HTTP endpoint).
5. Deploy the API:
   - Once your API configuration is complete, click on "Actions" and choose "Deploy API."
   - Select a deployment stage (e.g., "prod") and deploy your API.

Qaid-e-Awam University of Engineering Science and Technology Nawasbshah

Department of Software Engineering

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	**Components**	**Level of Achievement**			**Marks Achieved**
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date:** _____

Practical 15

Objective: Case study Open ended Lab

SaaS Deployment: Building and Deploying a Cloud-Based CRM System

Overview: This project aims to give students hands-on experience in deploying a Software as a Service (SaaS) application on a cloud computing platform. Students will work on setting up, configuring, and deploying a Cloud-based Customer Relationship Management (CRM) system. Through this project, students will gain practical insights into cloud infrastructure, scalability, security, and the deployment process of a SaaS application.

Objectives:

- Understand the architecture and components of a SaaS application.
- Gain practical experience in deploying a cloud-based CRM system.
- Configure and optimize the cloud environment for hosting the CRM system.
- Implement security measures to protect data and ensure compliance.
- Test the deployed CRM system for functionality, scalability, and performance.

Qaid-e-Awam University of Engineering Science and Technology Nawasbshah

Department of Software Engineering

Lab Rubrics

Course:	Cloud Computing	**Course Code:**	SW406
Student Name:		**Student Roll No.:**	
Semester, Year:	1st Semester, 4th Year	**Date:**	

SN	**Components**	**Level of Achievement**			**Marks Achieved**
		Proficient 0.25	**Acceptable 0.15**	**Unacceptable 0**	
1	**Problem Understanding and Analysis**	Students have carefully read and understood the problem. Students gives the idea to solve problem	Student understands problems and analysis in an acceptable way	Student don't understand problem.	
2	**Ability to use software/hardware**	Student was familiar with the software/hardware and required minimal help from the instructor to perform the experiment	Student demonstrated an ability to use the software/hardware but required assistance from the instructor	Student demonstrated little or no ability to perform experiment and required unreasonable amount of assistance from instructor	
3	**Completeness and accuracy**	The lab /tasks are complete and accurate in the context of implementation	The lab completion is just acceptable	The lab in its current state is not implementable	
4	**Q &A**	Able to explain design/program/simulation/fundamental concepts correctly	Able to explain some design/program/simulation/fundamental concepts	Unable to explain design/program/simulation/fundamental Concepts	
Total (1)					

Name & Signature of Teacher: _____ **Date:** _____